

2.5 PPT

2.5 项目演进路线 阶段二：Agent 工具调用基础设施 v2

运行演示

- 运行阶段二成品，以 Coding Agent 方式搜索自身代码，解释工具超时。
- 验证核心工程问题：
 - handler 完全忽略 `AbortSignal` 时，Runtime 能否按时返回 `TOOL_TIMEOUT` ?
 - 超时后重试和降级从哪里接上？
- 启动 Gateway 和 Runtime，配置环境变量。
- 使用 `plan` 模式，只读分析，不产生业务写入。
- 提示词明确要求证据组成：实现、调用方、测试、剩余风险。
- Agent 先 `grep` 符号，再分段 `read` 实现和测试。
- 关键命中：
 - `runWithDeadline` 调用，`tool.metadata.timeoutMs`，`TOOL_TIMEOUT` reject。
- 真正保证按时返回的是 `Promise.race`；`AbortController` 负责通知取消，两者并存。
- 重试条件：`ToolRuntimeError.retryable` 且工具声明 `idempotent`。
- 测试 handler 故意忽略 signal，超时 15ms，断言错误码和返回时间。
- 最终结论：
 - a. `Promise.race` 保证本地及时结束，`AbortController` 传递取消。
 - b. 超时标为 `retryable`，但重试还需幂等声明和 `maxRetries`。
 - c. fallback 同样经过 `deadline helper`。
 - d. 测试证明硬超时成立。
- 生产风险：本地超时不代表远端副作用停止，写操作需人工核对。
- 查看 Trace 验证执行过程。
- 后续按调用链拆解：模型入口 → 工具注册 → 参数校验 → 权限控制 → 人工审批 → 超时重试 → MCP 接入 → Trace。

一、先把阶段一 Gateway 接到 Agent

- Agent 只注册平台模型 `agent-default`，Base URL 指向 Gateway `/v1`。
- 供应商 Key 不进入 Node，只持有 Gateway Key。

代码块

```
1  const model: Model<"openai-completions"> = {
2    id: "agent-default",
3    provider: "phase-gateway",
4    api: "openai-completions",
5    baseUrl: config.gateway.baseUrl,
6    contextWindow: 1_000_000,
7    maxTokens: 4096,
8  };
```

- 平台模型名和供应商模型名分离，便于切换。
- 价格使用预算上界，LoopGuard 提前停止。
- Gateway 使用 LiteLLM 路由，先 Flash 后 Pro。
- LiteLLM `max_retries=0`，重试唯一负责方是 Gateway，工具重试由 Runtime 负责。
- 流式响应：首段发出前允许切换，之后中断返回 `STREAM_INTERRUPTED`。
- Gateway 记录模型调用：实际模型、Token、成本、延迟、尝试次数，不保存完整 Prompt。
- Prompt 统一管理，Agent 提交模板名称、版本和变量。

代码块

```
1  const response = await
    fetch(`${base}/v1/prompts/${encodeURIComponent(config.gateway.promptName)}/render`, {
2    method: "POST",
3    headers: { "content-type": "application/json", authorization: `Bearer ${apiKey}` },
```

```
4     body: JSON.stringify({
5         version: config.gateway.promptVersion,
6         variables: {
7             tenant_id: context.tenantId,
8             user_id: context.userId,
9             agent_name: context.agentName,
10        },
11    }),
12 });
```

- 变量严格匹配，拒绝多或少。

二、注册工具时，同时完成命名投影和能力快照

- 统一导入本地函数、Python 服务、MCP 工具到 ToolRegistry，生成快照。
- 注册完成内部身份、模型名称和治理元数据。

代码块

```
1 interface ToolMetadata {
2     internalName: string;
3     parameters: TSchema;
4     risk: "read" | "write" | "high";
5     permissions: string[];
6     requiresConfirmation: boolean;
7     timeoutMs: number;
8     maxRetries: number;
9     idempotent: boolean;
10    source: "local" | "python" | "mcp";
11 }
```

- 内部名称保留点号，模型名称转换点号为双下划线，注册时检查碰撞，启动失败。
- `ManagedTool` 包含 `handler` 和 `fallbackHandler`。

代码块

```
1 export interface ManagedTool {
2   metadata: ToolMetadata;
3   handler: ToolHandler;
4   fallbackHandler?: ToolHandler;
5 }
```

- 注册时维护两个索引：internalName 和 modelName。

代码块

```
1 export class ToolRegistry {
2   private readonly byInternalName = new Map<string, RegisteredTool>();
3   private readonly byModelName = new Map<string, RegisteredTool>();
4
5   register(tool: ManagedTool): RegisteredTool {
6     const internalName = tool.metadata.internalName;
7     if (this.byInternalName.has(internalName)) {
8       throw new Error(`Tool already registered: ${internalName}`);
9     }
10    const modelName = toModelToolName(internalName);
11    if (this.byModelName.has(modelName)) {
12      throw new Error(`Model tool-name collision: ${modelName}`);
13    }
14    const registered = { ...tool, modelName };
15    this.byInternalName.set(internalName, registered);
16    this.byModelName.set(modelName, registered);
17    return registered;
18  }
```

- 名称映射可逆、可预测，不自动截断。
- 快照按用户、Agent、租户、模式过滤。

代码块

```
1 snapshot(context: ExecutionContext, policy: PolicyEngine): RegisteredTool[] {
```

```
2     return this.list().filter((tool) => policy.isVisible(context,  
    tool.metadata.internalName));  
3 }
```

- Extension 在 `session_start` 动态注册快照工具，execute 调用 `runtime.invoke()`，确保同一执行路径。

代码块

```
1  for (const tool of registry.snapshot(context, policy)) {  
2      pi.registerTool({  
3          name: tool.modelName,  
4          label: tool.metadata.internalName,  
5          description: tool.metadata.description,  
6          parameters: tool.metadata.parameters,  
7          execute: (toolCallId, params) =>  
8              runtime.invoke({ context, toolCallId, modelName: tool.modelName, args:  
10         params })),  
9      });  
10 }
```

三、用 Pydantic 验证 Schema，在两个信任边界执行校验

- Python 业务服务用 Pydantic 定义输入模型。

代码块

```
1  class OrderStatusInput(BaseModel):  
2      model_config = ConfigDict(extra="forbid")  
3      order_id: str = Field(pattern=r"^[ORD-[A-Z]-[0-9]{4,12}$")
```

- 导出 JSON Schema，Node 读取，不手工维护两份契约。

代码块

```
1 schema = OrderStatusInput.model_json_schema()
```

- 保存到 `config/schemas/order.get_status.json`，CI 检查变更。
- 双层校验：pi 和 Runtime 拒绝非法调用；Python 接口再次校验，保护业务服务。
- `extra="forbid"` 防止静默忽略额外字段。
- Node 注册直接读取 Schema。

代码块

```
1 const schema = JSON.parse(  
2   readFileSync(  
3     "config/schemas/order.get_status.json",  
4     "utf-8",  
5   ),  
6 );  
7  
8 registry.register({  
9   metadata: {  
10    internalName: "order.get_status",  
11    description: "查询当前租户中的订单状态",  
12    parameters: schema,  
13    risk: "read",  
14    permissions: ["order:read"],  
15    requiresConfirmation: false,  
16    timeoutMs: 3000,  
17    maxRetries: 2,  
18    idempotent: true,  
19    source: "python",  
20  },  
21  
22  async handler({ args, context, signal }) {  
23    return callOrderService(args, context, signal);  
24  },  
25 });
```

- Schema 导出可重复执行： `python -m app.export_schemas` ，CI 检查 diff。
- 模型生成的参数先经 pi 校验，非法直接返回 `INVALID_ARGUMENT` 。
- `beforeToolCall` 修改参数后，Runtime 在执行前重新校验。

代码块

```
1  const prepared = runtime.prepare(request);
2  if ("tool" in prepared) {
3      return runtime.executePrepared(prepared);
4  }
```

四、把策略判断排在副作用之前，并固定判断顺序

- 执行顺序固定：查工具 → 校验参数 → 检查可见性 → 角色权限 → 运行模式 → 资源范围 → 人工确认。
- 执行上下文包含 `traceId`, `userId`, `tenantId`, `roles`, `agentName`, `mode`。

代码块

```
1  interface ExecutionContext {
2      traceId: string;
3      requestId: string;
4      userId: string;
5      tenantId: string;
6      roles: string[];
7      agentName: string;
8      mode: "plan" | "execute";
9  }
```

- 身份信息由系统提供，模型不能自声明。
- 配置角色权限和 Agent 白名单。

代码块

```
1  role_permissions:
2    supervisor:
3      - order:read
4      - ticket:create
5      - coffee:order
6
7  agent_tool_allowlist:
8    support-agent:
9      - order.get_status
10     - ticket.create
11     - mcp.my-coffee.*
```

- 权限合并：多角色取并集。
- 白名单支持精确匹配和命名空间匹配（*）。
- 策略主流程按顺序拒绝，每个拒绝有稳定 `code` 和 `reason`。

代码块

```
1  evaluate(
2    context: ExecutionContext,
3    tool: RegisteredTool,
4    args: Record<string, unknown>,
5  ): PolicyDecision {
6    if (!this.isVisible(context, tool.metadata.internalName)) {
7      return {
8        action: "deny",
9        code: "TOOL_NOT_VISIBLE",
10       reason: "工具不在当前 Agent 白名单中",
11      };
12    }
13
14    const permissions = this.permissionsFor(context);
15    const missing = tool.metadata.permissions.filter(
16      (permission) => !permissions.has(permission),
17    );
18
19    if (missing.length > 0) {
20      return {
21        action: "deny",
```



```

22     code: "RBAC_DENIED",
23     reason: `缺少权限: ${missing.join(", ")}` ,
24   };
25 }
26
27 if (
28   context.mode === "plan" &&
29   tool.metadata.risk !== "read"
30 ) {
31   return {
32     action: "deny",
33     code: "PLAN_MODE_DENIED",
34     reason: "plan 模式禁止写操作",
35   };
36 }
37
38 return {
39   action: "allow",
40   code: "OK",
41   reason: "策略检查通过",
42 };
43 }

```

- 资源范围判断示例（租户前缀）。

代码块

```

1  const orderId = args.order_id;
2  if (typeof orderId === "string") {
3    const prefixes = this.config.resourceTenantPrefixes[context.tenantId] ?? [];
4    if (!prefixes.some((prefix) => orderId.startsWith(prefix))) {
5      return {
6        action: "deny",
7        code: "RESOURCE_SCOPE_DENIED",
8        reason: "目标订单不属于当前租户",
9      };
10   }
11 }

```

- `plan` 模式禁止写操作，不能仅靠 Prompt 约束。

- 需确认的工具返回 `APPROVAL_REQUIRED`，创建持久化审批记录。

五、把人工审批设计成可恢复状态

- 审批不是对话布尔值，需持久化并绑定具体参数。
- 生成参数摘要（canonical JSON + SHA256）。

代码块

```
1  const digest = sha256(canonicalJson({
2    trace_id,
3    tool_call_id,
4    tool_name,
5    user_id,
6    tenant_id,
7    args,
8  }));
```

- 审批记录字段：id, trace_id, tool_call_id, tool_name, context_json, args_json, args_digest, status, expires_at, decided_by, created_at。
- Runtime 创建记录，返回 `APPROVAL_REQUIRED` 和 `approval_id`。
- `pi-agent-core` 通过 `shouldStopAfterTurn` 检测待审批，停止 Loop，返回 `AWAITING_APPROVAL`。
- 人通过 CLI 或 API 处理：list, decide (approve/reject), execute。
- 执行审批时重新完整校验：
 - 工具存在，名称映射未变。
 - 参数仍符合当前 Schema。
 - 用户权限和租户范围未变。
 - 参数摘要与批准时一致。
- 原子领取审批（SQL UPDATE 防止并发）。

代码块

```
1  claimApproval(id: string): boolean {
2    const result = db
3      .prepare(
4        `UPDATE approvals
5          SET status = 'executing'
6          WHERE id = ?
7            AND status = 'approved'
8            AND expires_at > ?`,
9      )
10     .run(id, new Date().toISOString());
11     return Number(result.changes) === 1;
12 }
```

- 状态流转：pending → approved → executing → consumed 或 rejected/failed。
- 高风险工具超时后不自动重放，需人工核对。
- 审批绑定用户、租户、工具名、Tool Call ID，不可重用。
- 模型不能产生有效审批，只能提示；程序保证持久化、防篡改、过期、单次消费、完整记录。

六、实现真正的硬超时

- 执行入口使用 `runWithDeadline` 包裹 handler。

代码块

```
1  const output = await runWithDeadline(
2    () =>
3      tool.handler({
4        toolCallId: prepared.toolCallId,
5        args: prepared.validatedArgs,
6        context: prepared.context,
7        signal: timeoutController.signal,
8      }),
9    tool.metadata.timeoutMs,
10    timeoutController,
```

11);

- `runWithDeadline` 实现: `Promise.race` + `controller.abort()`。

代码块

```
1  async function runWithDeadline<T>(  
2    work: () => Promise<T>,  
3    timeoutMs: number,  
4    controller: AbortController,  
5  ): Promise<T> {  
6    let timeout: ReturnType<typeof setTimeout> | undefined;  
7  
8    const deadline = new Promise<never>((_resolve, reject) => {  
9      timeout = setTimeout(() => {  
10        controller.abort();  
11        reject(new ToolRuntimeError("TOOL_TIMEOUT", "工具执行超时", true));  
12      }, timeoutMs);  
13    });  
14  
15    try {  
16      return await Promise.race([work(), deadline]);  
17    } finally {  
18      if (timeout) clearTimeout(timeout);  
19    }  
20  }
```

- 硬超时语义: 本地及时返回 + 尝试取消下游, 副作用不确定时不自动重放。
- 测试忽略 signal 的 handler, 断言超时错误码和返回时间 < 150ms。
- 重试仅幂等工具允许, `maxAttempts = idempotent ? maxRetries+1 : 1`。
- 错误分类: retryable vs non-retryable。
- 重试之间指数退避 (示例较短)。
- 重试次数进入 Trace。
- 降级返回明确不确定性 (如 `status: "unknown"`), 不能伪装成正常成功。
- fallback 同样经过硬超时, 成功码为 `FALLBACK_OK`。
- 模型重试 (Gateway) 与工具重试 (Runtime) 分层, 不共用计数。

- 写操作超时处理：返回未知，保留 Trace，用幂等键查询或人工核对。

七、接入 MCP 和治理链

- MCP 配置：my-coffee，streamablehttp，带 Token 环境变量。

代码块

```
1  mcp_servers:
2    - id: my-coffee
3      type: streamablehttp
4      url: https://gwmcp.lkcoffee.com/order/user/mcp
5      headers:
6        Authorization: Bearer ${LUCKIN_MCP_TOKEN}
7      trust: trusted
8      default_risk: high
9      permissions:
10     - coffee:order
```

- 环境变量展开，缺失直接失败。
- 创建 Streamable HTTP Transport 并连接 Client。
- 发现工具 (listTools)，每个远端工具转为内部 ManagedTool：

代码块

```
1  for (const remoteTool of discovered.tools) {
2    const internalName = `mcp.${server.id}.${remoteTool.name}`;
3
4    registry.register({
5      metadata: {
6        internalName,
7        description: remoteTool.description ?? `MCP tool ${remoteTool.name}`,
8        version: "mcp",
9        parameters: remoteTool.inputSchema as TSchema,
10       risk: server.defaultRisk,
```

```

11     permissions: server.permissions,
12     requiresConfirmation: server.defaultRisk !== "read",
13     timeoutMs: 10_000,
14     maxRetries: server.defaultRisk === "read" ? 1 : 0,
15     idempotent: server.defaultRisk === "read",
16     source: "mcp",
17 },
18
19 async handler({ args, signal }) {
20     const result = await client.callTool(
21         {
22             name: remoteTool.name,
23             arguments: args,
24         },
25         undefined,
26         { signal, timeout: 10_000 }
27     );
28
29     if (result.isError) {
30         throw new ToolRuntimeError("MCP_TOOL_ERROR",
normalizeMcpText(result.content), false);
31     }
32     if ("structuredContent" in result && result.structuredContent) {
33         return result.structuredContent;
34     }
35     return { content: normalizeMcpText(result.content), server_id: server.id
};
36 },
37 });
38 }

```

- 模型名称映射： `mcp__my-coffee__create_order` ，Trace 区分来源。
- MCP 工具经过完整治理链：校验、白名单、RBAC、租户、确认、执行、审计。
- 默认高风险，需权限和审批。
- 取消信号传递给 MCP SDK。
- 输出规范化，限制大小和脱敏。
- 工具 Schema 变化需告警，高风险工具重新确认。
- 测试分本地（stdio）和正式（live）两种，无 Token 时失败，不能伪造。

八、用 Audit 记录责任，用 Trace 还原执行链

- Agent Run 开始时创建统一 `traceId`，所有事件携带。
- 工具调用有独立的 `toolCallId`。
- Audit 记录责任：who, when, what, on which resource。

代码块

```
1 interface ToolAuditRecord {
2   trace_id: string;
3   tool_call_id: string;
4   tool_name: string;
5   tool_version: string;
6   user_id: string;
7   tenant_id: string;
8   agent_name: string;
9   risk_level: string;
10  status: string;
11  code: string;
12  started_at: string;
13  finished_at: string;
14  latency_ms: number;
15  input_redacted: unknown;
16  output_redacted?: unknown;
17  error_code?: string;
18 }
```

- 策略拒绝、成功、失败均记录。
- 脱敏在写入前完成，使用配置敏感字段 + 通用正则。

代码块

```
1 export function redact(value: unknown, configured: string[] = []): unknown {
2   const sensitive = new Set(configured.map(item => item.toLowerCase()));
3   if (value && typeof value === "object") {
4     return Object.fromEntries(
5       Object.entries(value as Record<string, unknown>).map(([key, item]) => [
```

```
6         key,  
7         sensitive.has(key.toLowerCase()) || GENERIC_SENSITIVE_KEYS.test(key) ?  
    "***" : redact(item, configured),  
8     ])  
9     );  
10    }  
11    return value;  
12 }
```

- Trace 记录 pi 事件（`message_end`，`turn_end`，`tool_execution_start/end`）和 Runtime 事件（`approval_requested`，`completed`）。
- 查看 Trace: `npm run cli -- trace <trace-id>`。
- Audit 和 Trace 分表存储，保留周期不同。
- 补录 pi 即时失败事件（`wasAudited()` 防重复）。
- 审计失败不影响业务结果（分别处理）。
- 验收场景覆盖：参数错误、权限拒绝、高风险审批、超时、MCP 调用等。

本章小结

- 统一模型调用（LiteLLM Gateway）+ pi 的 Provider/Tool 注册。
 - 本地函数和 MCP 远程工具统一纳入 ToolRegistry，标准化参数、风险、权限。
 - 调用前：Pydantic/JSON Schema 校验，身份、租户、角色、白名单、风险检查，高风险人工确认（参数摘要防篡改）。
 - 调用中：统一 Runtime，具备超时、有限重试、幂等键、降级策略。
 - MCP 工具不能绕过治理链。
 - Audit（责任）和 Trace（过程）完整，脱敏前置。
 - LoopGuard 控制预算和终止，TDD 测试公开行为。
 - 最终形成了可接入真实业务、可审计、可运维的 Agent 执行底座。
-

参考资料与实现依据

- pi 项目源码
- pi Agent Core
- pi AI
- pi Extensions
- pi Security
- LiteLLM 文档
- LiteLLM Routing
- LiteLLM DeepSeek Provider
- DeepSeek Tool Calls
- MCP TypeScript SDK
- MCP Tools 规范